```
gem "rspec", git: "git@github.com:rspec/rspec-metagem.git"
```

Any of the Git access versions will work here, assuming that you have the appropriate access to the remote Git server.

If the gem is hosted on GitHub, there's a shortcut:

```
gem "rspec", github: "rspec/rspec"
```

And Bundler will use your GitHub access to install the gem. (There's a similar option for BitBucket.) In both cases, you can further specify which part of the repo you want with a branch:, tag:, or ref: option.

When using a git source, you must bundle update to change the git commit that you are pointing to.

If you are working on the gem yourself locally, you can specify a path option that points to the top-level directory of the gem (specifically, the location of the .gemspec file) relative to the location of the Gemfile:

```
gem "rspec", path: "../development/rspec"
```

Unlike a git source, when you use a path, Bundler executes the local source directly each time, so you do not need to bundle update to get the newest version of a gem you are working on locally.

Gemfile Groups

Another way to specify more information about a gem is with the groups feature. A group is a collection of gems within a Gemfile that can be included or not-included together when Bundler is used. Commonly, this feature is used to limit what gems are loaded based on environment or context. You may have certain gems that are only useful in production, or only needed when running tests.

There are two ways to attach gems to groups.

You can specify one or more groups for a gem by using the group: option inside the gemfile; the argument is either a single symbol or an array of symbols representing the group or groups that the gem is a part of:

```
gem "rspec", group: [:development, :test]
```

We've assigned RSpec to the :development, and :test groups. The group names are arbitrary, but it's conventional to match them to environment names when possible.

Alternately you can use the block form of group. This is shorter in the common case where you have several gems belonging to the same group or groups.

```
group :development do
  gem "standardrb"
  gem "debug"
end

group :development, :test do
  gem "rspec"
end
```